

HRG Grouper Analysis Run Book

This document serves as a run book for the HRG Grouper Analysis project. It Completes the steps to set up the environment, run the analysis, and interpret the results. This can either be done with real data by an NHS organization, or with the test data provided by the NHS England's Casemix Office. Out of the box this uses the test data.

0.0 Setup

0.1 Install requirements

```
In [ ]: import Utils.install_requirements

Utils.install_requirements.install_requirements()
```

0.2 Download and Extract the Test Data

```
In [ ]: import requests
import os
import zipfile
import shutil
from Utils.constants import DATA_FILE_FOLDER, SAMPLE_DATA_FILE, RAW_FILE_FOLDER

# URL of the zip file
url = 'https://digital.nhs.uk/binaries/content/assets/website-assets/services/national-casemix-office/hr4-2024-25-1c
zip_path = os.path.join(DATA_FILE_FOLDER, 'test_data.zip')
target_file_path_and_name = 'HRG4+ 202425 Local Payment Grouper Test Data and Expected Results v1.0/APC/HRG4+ 202425

# Download the file
response = requests.get(url, stream=True)
response.raise_for_status()
with open(zip_path, 'wb') as f:
    for chunk in response.iter_content(chunk_size=8192):
        f.write(chunk)
```

```

with zipfile.ZipFile(zip_path, 'r') as f:
    f.extract(target_file_path_and_name, path=DATA_FILE_FOLDER)

shutil.move(
    os.path.join(DATA_FILE_FOLDER, target_file_path_and_name),
    os.path.join(RAW_FILE_FOLDER, SAMPLE_DATA_FILE)
)

os.remove(zip_path)

directory_to_remove = os.path.join(DATA_FILE_FOLDER, target_file_path_and_name.split('/')[0])
shutil.rmtree(directory_to_remove)

# Print the path where the data has been downloaded
print(f'Test data downloaded to: {RAW_FILE_FOLDER}/{SAMPLE_DATA_FILE}')

```

0.3 Download the HRG Grouper application

```

In [ ]: import requests
import os
import zipfile
import shutil
import tkinter as tk
from tkinter import messagebox
from Utils.constants import DATA_FILE_FOLDER, SAMPLE_DATA_FILE

def init_tk_root():
    '''
        Initializes a hidden Tkinter root window.
    '''

    root = tk.Tk()
    root.attributes('-topmost', True)
    root.withdraw()
    return root

# URL of the zip file
url = 'https://digital.nhs.uk/binaries/content/assets/website-assets/services/national-casemix-office/hrg4-2024-25-1c
zip_file = 'hrg_grouper.zip'

```

```

target_file_path_and_name = 'HRG4+ 202425 Local Payment Grouper/HRGGrouperSetup.exe'
installer_name = target_file_path_and_name.split('/')[1]

# Download the file
response = requests.get(url, stream=True)
response.raise_for_status()
with open(zip_file, 'wb') as f:
    for chunk in response.iter_content(chunk_size=8192):
        f.write(chunk)

# unzip the file
with zipfile.ZipFile(zip_file, 'r') as f:
    f.extract(target_file_path_and_name)

shutil.move(target_file_path_and_name, installer_name)

# Remove the zip file after extraction
os.remove(zip_file)
directory_to_remove = target_file_path_and_name.split('/')[0]
shutil.rmtree(directory_to_remove)

# Print the path where the installer has been downloaded
print(f'HRG Application Installer: ./{installer_name}')

root = init_tk_root()
run_installer = messagebox.askyesno(
    "Run Installer",
    f"Would you like to run the HRG Grouper installer now?\n{installer_name}",
    parent=root
)

if run_installer:
    # Use os.startfile with 'runas' to request elevation
    try:
        os.startfile(installer_name, 'runas')
    except Exception as e:
        messagebox.showerror("Error", f"Failed to run the installer: \n{e}", parent=root)

root.destroy()

```

0.4 Find and add the HRG Grouper executable to the PATH

```

In [ ]: import os
import tkinter as tk
from tkinter import messagebox, filedialog

# Default install location
default_path = r'C:/Program Files/NHS England/HRG4+ 2024_25 Payment Grouper/HRGGrouper.exe'

def set_grouper_exe_env(path):
    """
    Sets the GROUPER_EXE environment variable and updates the .env file.
    """
    os.environ['GROUPER_EXE'] = path
    lines = []
    # Check that it isn't already set in .env
    if os.path.exists('.env'):
        with open('.env', 'r') as f:
            lines = f.readlines()
    with open('.env', 'w') as f:
        found = False
        for line in lines:
            if line.startswith('GROUPER_EXE='):
                f.write(f'GROUPER_EXE="{path}"\n')
                found = True
            else:
                f.write(line)
        if not found:
            f.write(f'GROUPER_EXE="{path}"\n')

def get_grouper_exe_from_env():
    """
    Reads the GROUPER_EXE environment variable from the .env file.
    Returns the path if found.
    """
    if not os.path.exists('.env'):
        return ''
    with open('.env', 'r') as f:
        for line in f:

```

```

        if line.startswith('GROUPER_EXE='):
            # get the path after the '=' and strip quotes
            return line.split('=', 1)[1].strip().strip('"').strip("'")
    return ''

def init_tk_root():
    """
    Initializes a hidden Tkinter root window.
    """
    root = tk.Tk()
    root.attributes('-topmost', True)
    root.withdraw()
    return root

def prompt_for_grouper_exe():
    """
    Prompts the user to select the HRG Grouper executable file.
    If the user selects a file, it sets the GROUPER_EXE environment variable.
    """
    root = init_tk_root()
    while True:
        file_path = filedialog.askopenfilename(
            title='Select HRGGrouperc.exe',
            filetypes=[('Executable files', '*.exe')],
            parent=root)
        if not file_path:
            messagebox.showwarning("No file selected", "No file was selected.", parent=root)
            break
        if not file_path.lower().endswith("hrggrouperc.exe"):
            retry = messagebox.askyesno(
                "Confirm file",
                f"The application by default is called 'HRGGrouperc.exe'.\nYou selected: {os.path.basename(file_path)}",
                parent=root)
            if not retry:
                continue
            set_grouper_exe_env(file_path)
            break
    root.destroy()

def handle_grouper_not_found(title, message):
    """
    Prompts the user with a message box when the HRG Grouper application is not found.
    """

```

```

...
root = init_tk_root()
messagebox.showinfo(title, message)
prompt_for_grouper_exe()
root.destroy()

grouper_exe_path = get_grouper_exe_from_env()
if grouper_exe_path:
    if not os.path.exists(grouper_exe_path):
        handle_grouper_not_found(
            "Not Found",
            f"HRG Grouper application path in .env does not exist: {grouper_exe_path}\n" \
            f"Please provide the path to HRGGrouperc.exe.")
    else:
        print(f"Found HRG Grouper application path in .env: {grouper_exe_path}")
        set_grouper_exe_env(grouper_exe_path)
elif os.path.exists(default_path):
    print(f"Found HRG Grouper application at default location: {default_path}")
    set_grouper_exe_env(default_path)
else:
    root = init_tk_root()
    answer = messagebox.askquestion(
        "HRG Grouper Application",
        "Have you installed the HRG Grouper application?",
        icon='question', parent=root)
    if answer == "no":
        msg = tk.Toplevel(root)
        msg.title("Download Required")
        tk.Label(msg, text="Please download and install the HRG Grouper application.").pack(padx=20, pady=20)
        msg.after(10000, msg.destroy)
        root.wait_window(msg)
        root.destroy()
    else:
        handle_grouper_not_found(
            "Not Found",
            "We were not able to find the HRG Grouper application at the default location.\n" \
            "Please provide the path to HRGGrouperc.exe.")

```

0.5 Build tariff key-value store

```
In [ ]: from tariff_kv_store import get_tariff_kv_store
try:
    _ = get_tariff_kv_store()
    print("Tariff key-value store initialized successfully.")
except Exception as e:
    print(f"Error initializing tariff key-value store: {e}")
    # If the tariff kv store is not initialized, we can try to initialize it again
```

0.6 Compile C Utilities

```
In [ ]: # Check if clang is installed
import shutil
import subprocess

clang_path = shutil.which("clang")
if clang_path:
    print(f"Clang found at: {clang_path}")
    c_utils_dir = "./C_Utils"
    c_library = "csv_utils.c"

    c_files = [
        f for f in os.listdir(c_utils_dir)
        if f.endswith(".c") and f != c_library
    ]

    for c_file in c_files:
        c_file_path = os.path.join(c_utils_dir, c_file)
        c_library_path = os.path.join(c_utils_dir, c_library)
        exe_name = os.path.splitext(c_file)[0] + ".exe"
        exe_path = os.path.join(c_utils_dir, exe_name)
        subprocess.run([
            "clang", "-Wall", "-Wextra", "-Werror", "-g", "-O0", c_library_path,
            c_file_path, "-o", exe_path
        ], check=True)
        print(f"Compiled {c_file} to {exe_name}")
else:
    print("Clang is not installed. Please install clang or compile manually.")
```

1.0 Check for and review invalid primary data elements

```
In [ ]: from Utils.run_validation import run_validation
_ = run_validation(report_mode=True)
```

1.1 Replace invalid primary data elements with valid values

```
In [ ]: from os import path, remove
from shutil import copyfile
from Utils.run_validation import run_validation
from Utils.constants import RAW_FILE_FOLDER, SAMPLE_DATA_FILE

file, df = run_validation(report_mode=False)
if not file:
    raise FileNotFoundError("No file was generated by the validation process.")
output_path = path.join(RAW_FILE_FOLDER, SAMPLE_DATA_FILE)
try:
    copyfile(file, output_path)
except Exception as e:
    raise IOError(f"Failed to copy file to {output_path}: {e}")
try:
    remove(file)
except Exception as e:
    raise IOError(f"Failed to remove temporary file {file}: {e}")

print(f"Overwritten {output_path} with validated data.")
```

1.2 Run the input data element impact assessment probes

```
In [1]: from run_multiple_probes import run_all_probes
# Run all probes together
try:
    run_all_probes()
    print("All probes executed successfully.")
```

```
except Exception as e:
    print(f"Error running probes: {e}")
```

n (non-null secondary diagnoses): 100%|██████████| 14/14 [08:57<00:00, 38.37s/it]
Comparison results saved to ./data/processed/multiple_probes_results.csv
All probes executed successfully.

1.3 Review the impact assessment results

```
In [ ]: import pandas as pd

probe_results_file = 'data/processed/multiple_probes_results.csv'

# Load the probe results
print("Loading probe results...")
df = pd.read_csv(probe_results_file, low_memory=False)

df = df[['PROVSPNO', 'Probe', 'ProbeValue', 'Match']]

print(f"Loaded {len(df)} rows from probe results file")
print(f"Columns: {list(df.columns)}")

print("\nCalculating match ratios by Probe and ProbeValue...")

# Group by Probe and ProbeValue and calculate match ratios
match_summary = (
    df.groupby(['Probe', 'ProbeValue'], as_index=False, dropna=False)
      .agg(
        Total_Records=('Match', 'count'),
        True_Matches=('Match', 'sum'),
        Match_Percentage=('Match', lambda x: (x.sum() / len(x)) * 100)
      )
      .round(2)
)

match_summary.columns = ['Probe', 'ProbeValue', 'Total_Records', 'True_Matches', 'Match_Percentage']
match_summary = match_summary.reset_index()
display(match_summary)
```

Loading probe results...

Loaded 567837 rows from probe results file

Columns: ['PROVSPNO', 'Probe', 'ProbeValue', 'Match']

Calculating match ratios by Probe and ProbeValue...

	index	Probe	ProbeValue	Total_Records	True_Matches	Match_Percentage
0	0	AdmitMethod	ELECTIVE_BOOKED	490	490	100.0
1	1	AdmitMethod	ELECTIVE_PLANNED	490	490	100.0
2	2	AdmitMethod	ELECTIVE_WAITING_LIST	490	490	100.0
3	3	AdmitMethod	EMERGENCY_BABY_HOME	490	490	100.0
4	4	AdmitMethod	EMERGENCY_BED_BUREAU	490	490	100.0
...	...	...	...	...	...	...
584	584	TreatmentFunctionCode	UROLOGY	490	490	100.0
585	585	TreatmentFunctionCode	VASCULAR_PHYSIOLOGY	490	490	100.0
586	586	TreatmentFunctionCode	VASCULAR_SURGERY	490	490	100.0
587	587	TreatmentFunctionCode	WELL_BABY	490	490	100.0
588	588	NaN	NaN	0	0	0.0

589 rows × 6 columns

```
In [10]: # Separate probes with 100% match from others
# Find probes where all ProbeValue combinations have 100% match
probe_group = match_summary.groupby('Probe')['Match_Percentage'].min().reset_index()
perfect_probe_names = probe_group[probe_group['Match_Percentage'] == 100.0]['Probe']
perfect_matches_probe = match_summary[match_summary['Probe'].isin(perfect_probe_names)]

perfect_matches = match_summary[(match_summary['Match_Percentage'] == 100.0) & (~match_summary['Probe'].isin(perfect_
partial_matches = match_summary[match_summary['Match_Percentage'] != 100.0]

print(f"\nMatch Ratio Analysis:")

if len(partial_matches) > 0:
```

```

partial_summary_df = partial_matches[['Probe', 'ProbeValue', 'Total_Records', 'True_Matches', 'Match_Percentage']]
# Convert True_Matches and Total_Records to int for calculation
partial_summary_df['True_Matches'] = partial_summary_df['True_Matches'].astype(int)
partial_summary_df['Total_Records'] = partial_summary_df['Total_Records'].astype(int)
partial_summary_df['False_Matches'] = partial_summary_df['Total_Records'] - partial_summary_df['True_Matches']
print("Partial match summary DataFrame:")
display(partial_summary_df)
else:
    partial_summary_df = None

if len(perfect_matches) > 0:
    print("ProbeValues with 100% match:")
    partial_perfect_probes = perfect_matches['Probe'].unique()
    print(f"The following {len(partial_perfect_probes)} probe(s) had 100% match across a subset of their probe values")
    perfect_summary_df = (
        perfect_matches[['Probe', 'ProbeValue', 'Total_Records', 'True_Matches', 'Match_Percentage']]
        .copy()
    )
    perfect_summary_df['True_Matches'] = perfect_summary_df['True_Matches'].astype(int)
    perfect_summary_df['Total_Records'] = perfect_summary_df['Total_Records'].astype(int)
    perfect_summary_df['False_Matches'] = perfect_summary_df['Total_Records'] - perfect_summary_df['True_Matches']
    display(perfect_summary_df)

if len(perfect_matches_probe) > 0:
    print("Probes with 100% match:")
    perfect_probes = perfect_matches_probe['Probe'].unique()
    print(f"The following {len(perfect_probes)} probe(s) had 100% match across all of their probe values:")
    # Create a DataFrame summarizing perfect matches
    perfect_probes_summary_df = (
        perfect_matches_probe[['Probe', 'ProbeValue', 'Total_Records', 'True_Matches', 'Match_Percentage']]
        .copy()
    )
    perfect_probes_summary_df['True_Matches'] = perfect_probes_summary_df['True_Matches'].astype(int)
    perfect_probes_summary_df['Total_Records'] = perfect_probes_summary_df['Total_Records'].astype(int)
    perfect_probes_summary_df['False_Matches'] = perfect_probes_summary_df['Total_Records'] - perfect_probes_summary_df['True_Matches']
    display(perfect_probes_summary_df)

print(f"\nSummary:")
print(f"- Total unique Probe-ProbeValue combinations: {len(match_summary)}")
print(f"- Combinations with 100% match: {len(perfect_matches)}")
print(f"- Combinations with partial match: {len(partial_matches)}")

```

```
# Store results for further analysis
probe_results = match_summary
```

Match Ratio Analysis:

Partial match summary DataFrame:

	Probe	ProbeValue	Total_Records	True_Matches	Match_Percentage	False_Matches
20	AdmitSource	BORN_IN_OR_ON_WAY_TO_HOSPITAL	490	489	99.80	1
21	AdmitSource	COURT	490	489	99.80	1
22	AdmitSource	FOSTER_CARE	490	489	99.80	1
23	AdmitSource	LOCAL_AUTHORITY	490	489	99.80	1
24	AdmitSource	NHS_CARE_HOME	490	489	99.80	1
...	...	...	...	...	...	...
451	TreatmentFunctionCode	DIAGNOSTIC_IMAGING	490	487	99.39	3
495	TreatmentFunctionCode	NUCLEAR_MEDICINE	490	489	99.80	1
539	TreatmentFunctionCode	PAEDIATRIC_PAIN_MANAGEMENT	490	480	97.96	10
549	TreatmentFunctionCode	PAIN_MANAGEMENT	490	480	97.96	10
588	NaN	NaN	0	0	0.00	0

261 rows × 6 columns

ProbeValues with 100% match:

The following 2 probe(s) had 100% match across a subset of their probe values:

	Probe	ProbeValue	Total_Records	True_Matches	Match_Percentage	False_Matches
40	Combinations	13	35	35	100.0	0
41	Combinations	14	1	1	100.0	0
408	TreatmentFunctionCode	ACUTE_INTERNAL_MEDICINE	490	490	100.0	0
409	TreatmentFunctionCode	ADDICTION	490	490	100.0	0
410	TreatmentFunctionCode	ADULT_CYSTIC_FIBROSIS	490	490	100.0	0
...	...	...	...	...	...	...
583	TreatmentFunctionCode	UROLOGICAL_PHYSIOLOGY	490	490	100.0	0
584	TreatmentFunctionCode	UROLOGY	490	490	100.0	0
585	TreatmentFunctionCode	VASCULAR_PHYSIOLOGY	490	490	100.0	0
586	TreatmentFunctionCode	VASCULAR_SURGERY	490	490	100.0	0
587	TreatmentFunctionCode	WELL_BABY	490	490	100.0	0

178 rows × 6 columns

Probes with 100% match:

The following 6 probe(s) had 100% match across all of their probe values:

	Probe	ProbeValue	Total_Records	True_Matches	Match_Percentage	False_Matches
0	AdmitMethod	ELECTIVE_BOOKED	490	490	100.0	0
1	AdmitMethod	ELECTIVE_PLANNED	490	490	100.0	0
2	AdmitMethod	ELECTIVE_WAITING_LIST	490	490	100.0	0
3	AdmitMethod	EMERGENCY_BABY_HOME	490	490	100.0	0
4	AdmitMethod	EMERGENCY_BED_BUREAU	490	490	100.0	0
...	...	...	...	...	...	...
271	Sex	FEMALE	490	490	100.0	0
272	Sex	INDETERMINATE	490	490	100.0	0
273	Sex	MALE	490	490	100.0	0
274	Sex	NOT_KNOWN	490	490	100.0	0
275	Sex	NOT_RECORDED	490	490	100.0	0

150 rows × 6 columns

Summary:

- Total unique Probe-ProbeValue combinations: 589
- Combinations with 100% match: 178
- Combinations with partial match: 261

1.4 Summarize Match Rates at the Probe Level

```
In [16]: import pandas as pd

# Group by Probe and calculate match rates and counts
probe_level_summary = (
    match_summary.groupby('Probe', as_index=False)
    .agg(
        Total_Records=('Total_Records', 'sum'),
        True_Matches=('True_Matches', 'sum')
    )
)
```

```

)

# Convert True_Matches to int if it's not already
probe_level_summary['True_Matches'] = probe_level_summary['True_Matches'].astype(int)
probe_level_summary['Total_Records'] = probe_level_summary['Total_Records'].astype(int)

probe_level_summary['Match_Percentage'] = (
    probe_level_summary['True_Matches'] / probe_level_summary['Total_Records'] * 100
).round(2)
probe_level_summary['False_Matches'] = probe_level_summary['Total_Records'] - probe_level_summary['True_Matches']

display(probe_level_summary)

```

	Probe	Total_Records	True_Matches	Match_Percentage	False_Matches
0	AdmitMethod	9800	9800	100.00	0
1	AdmitSource	7840	7807	99.58	33
2	Combinations	286087	19783	6.92	266304
3	DischargeDestination	11760	11760	100.00	0
4	DischargeMethod	3430	3430	100.00	0
5	EpisodeDuration	47040	36579	77.76	10461
6	MainSpecialty	43120	43120	100.00	0
7	PatientClassification	2940	2940	100.00	0
8	Sex	2450	2450	100.00	0
9	StartAge	64680	51872	80.20	12808
10	TreatmentFunctionCode	88200	88176	99.97	24